

Hefest in Context

Mapping the engine's code onto
Game Engine Architecture (3rd ed.) and the *Vulkan Specification* (1.4.361)

A short exploration, not an audit

2026-09-01

Hefest is a ~18 kLOC C engine with a Vulkan renderer. Almost every non-obvious decision in it is a decision that one of these two documents already argues for — sometimes because the code was written from them, sometimes because both arrived at the same place independently. This report picks six such decisions, states the concept as the source states it, then shows the code that implements it and how far that implementation goes.

Two documents of very different character are being used here. *Game Engine Architecture* is a survey of engineering practice: it says what an engine usually contains and why. The *Vulkan Specification* is a contract: it says what the driver guarantees and what the application must guarantee back. The interesting result is how often the second document's contract turns out to be the reason the first document's advice is good advice.

Contents

1	Introduction and method	1
1.1	What is being compared	1
1.2	How to read the citations	1
1.3	Vocabulary	1
2	Subsystem lifecycle and explicit sizing	2
2.1	The problem both documents solve	2
2.2	The twist Vulkan contributes	2
2.3	The cost of the module-static state pointer	3
3	Memory ownership	4
3.1	Level 1: tagged host allocation	4
3.2	Level 2: the linear allocator	4
3.3	Level 3: device memory and the staging pattern	5
4	The frame loop and CPU/GPU pipelining	6
4.1	Framework, not library	6
4.2	Where the GPU changes the picture	6
4.3	Resize as a generation counter	8
5	Resources, references and the asset pipeline	9
5.1	Reference counting by name	9
5.2	Generations as a staleness signal	9
5.3	The asset pipeline, in miniature	10
5.4	How this reaches the GPU	10
6	Abstraction seams	12
6.1	The platform layer	12
6.2	The renderer frontend/backend split	12
6.3	Vulkan's own seam	12
6.4	Instrumentation as an engine concern	13
7	What the sources point at next	14
7.1	A closing observation	14

1 Introduction and method

1.1 What is being compared

Hefest is a shared library (`libengine.so / engine.dll`) consumed by an executable, `testbed`, which supplies a game struct of function pointers. The engine owns `main()`. Inside it are the usual layers: a platform abstraction over Win32 and XCB/X11, a set of core subsystems (logging, memory, events, input, clock, strings), containers, a math library, a renderer split into an API-agnostic frontend and a Vulkan backend, and — most recently — resource systems for textures and materials.

At the time of writing, the renderer draws one hardcoded textured quad with a free-flying camera. That is a small output for the amount of machinery behind it, and the gap is the point: most of the code exists to establish structure that only pays off later. That makes it good material for a comparison against two documents that are largely *about* structure.

1.2 How to read the citations

Each section opens with a box naming the relevant sections of both sources plus the files in the engine that implement the concept. Inline, [GEA §6.1](#) means *Game Engine Architecture* section 6.1 and [VkSpec §7.3](#) means chapter/section 7.3 of the Vulkan specification. Book page numbers are given where a specific passage is quoted. Code references use the repo-relative path and line number, e.g. `engine/src/core/application.c:71`.

1.3 Vocabulary

The report assumes a general software-engineering background and no graphics background. Three terms recur and are worth fixing up front:

Host and device In Vulkan’s language the **host** is the CPU running your program and the **device** is the GPU. They run asynchronously. Nearly every awkward-looking thing in a Vulkan renderer exists to manage that asynchrony explicitly, because the API does almost nothing implicitly.

Descriptor A descriptor is a handle-shaped record that tells a shader where to find a resource — “the uniform buffer for this frame lives at this offset in this buffer”, “the texture to sample lives in this image with this sampler”. A **descriptor set** is a group of them bound together. Think of it as a small, explicitly-laid-out argument struct that the GPU reads.

Subsystem In *Game Engine Architecture*’s usage, a long-lived engine-global module with explicit start-up and shut-down — a singleton in the “one per process” sense, not necessarily in the Gang-of-Four sense.

2 Subsystem lifecycle and explicit sizing

Game Engine Architecture	GEA §6.1, p.417 Subsystem Start-Up and Shut-Down; GEA §6.1.2, p.419 A Simple Approach That Works
Vulkan Specification	VkSpec §4.2 Instances; VkSpec §5.1 Physical Devices — the enumerate/query calling convention
Hefest	core/application.c:59-146, every *_initialize in core/, platform/, renderer/, systems/

2.1 The problem both documents solve

Gregory opens chapter 6 by ruling out the language’s own answer. C++ constructs global and static objects before main(), and:

“these constructors are called in a totally unpredictable order [...] Clearly this behavior is not desirable for initializing and shutting down the subsystems of a game engine, or indeed any software system that has interdependencies between its global objects.” [GEA §6.1.1, p.418](#)

His recommendation is deliberately unclever: give each subsystem explicit startUp() and shutDown() functions, make the constructor and destructor do nothing, and call them in a hand-written order from main(). He calls this the “brute-force” approach and defends it on the grounds that the order is then visible in one place and trivially adjustable [GEA §6.1.2, p.421](#).

Hefest is written in C, so the static-initialization hazard does not arise in the same form — but the underlying problem does. Subsystem A needs subsystem B running before it can start, and nothing in the language will enforce or even document that. application_create is the one place where the ordering lives:

```
// engine/src/core/application.c
event_system_initialize(...);    // first: input depends on it
memory_system_initialize(...);   // tracker only; allocation works without it
initialize_logging(...);         // needs filesystem + platform console
input_system_initialize(...);
platform_system_startup(...);    // creates the window
renderer_system_initialize(...); // needs the window for surface creation
texture_system_initialize(...);   // needs the renderer for create_texture
```

That is Gregory’s chapter 6.1 almost verbatim, including his tolerance for imperfection: he notes that shutting down in an order that is not exactly the reverse of start-up is a “minor disadvantage” not worth losing sleep over. The engine’s application_shutdown does in fact deviate slightly from strict reverse order and is none the worse for it.

2.2 The twist Vulkan contributes

Where Hefest departs from the book is in **how** each subsystem gets its memory. Every *_initialize in the engine is called twice:

```
u64 size;
subsystem_initialize(&size, 0);           // 1. how big is your state?
void* mem = linear_allocator_allocate(    // 2. carve that much from the block
    &app->systems_allocator, size);
subsystem_initialize(&size, mem);         // 3. initialize, using that memory
```

and inside:

```
b8 subsystem_initialize(u64* memory_requirement, void* state) {
    *memory_requirement = sizeof(subsystem_state);
    if (state == 0) { return true; } // size-query mode
    state_ptr = state;
    /* ... real init ... */
}
```

```

    return true;
}

```

This is Vulkan’s enumeration convention lifted wholesale into engine code. The specification’s wording for `vkEnumeratePhysicalDevices` is the canonical instance of it:

“If `pPhysicalDevices` is `NULL`, then the number of physical devices available is returned in `pPhysicalDeviceCount`. Otherwise, `pPhysicalDeviceCount` must point to a variable set by the application to the number of elements in the `pPhysicalDevices` array...” [VkSpec §5.1](#)

The same shape appears dozens of times across the specification — `vkEnumerateInstanceLayerProperties`, `vkGetPhysicalDeviceQueueFamilyProperties`, `vkGetSwapchainImagesKHR`, all of which Hefest calls. The reason Vulkan does it is that the API refuses to allocate on the application’s behalf: the caller must own the storage, so the caller must first be told how much storage to own.

Hefest adopts the idiom for exactly the same reason at a different scale. A subsystem that does not allocate its own state cannot impose an allocator on the rest of the engine, cannot fragment the heap, and cannot outlive the block it was given. `application_create` allocates one 64 MiB linear allocator (`application.c:71`) and carves every subsystem’s state out of it in sequence. The engine ends up with all of its long-lived state in one contiguous span, which is a cache-locality argument Gregory makes independently in [GEA §6.2.1](#).

The two ideas — Gregory’s explicit ordering and Vulkan’s explicit sizing — compose neatly. The result is a start-up sequence where both **when** a subsystem comes up and **where its memory lives** are decided by the caller, in one readable function.

2.3 The cost of the module-static state pointer

Each subsystem stores the block it was handed in a translation-unit-static pointer:

```
static subsystem_state* state_ptr;
```

This is the concession that makes the pattern ergonomic — no `self` parameter threaded through every call — and it is exactly the singleton that Gregory’s chapter assumes. It costs one instance per process. There is no second renderer for a tool window, no per-thread event queue, no re-entrant re-initialization. For a single-game engine that is a reasonable trade, and the docs record it as deliberate.

Where it stops. The pattern has no idempotence guard: calling the **real** `init` twice overwrites `state_ptr` and leaks whatever the first call owned. `application_create` guards itself (`application.c:60`) but individual subsystems do not. Neither does `init` failure roll back the subsystems that already came up — `application_create` returns `false` and the process exits, so the block is reclaimed by the OS rather than by the engine. Both are the kind of thing Gregory’s chapter would flag if the engine ever needed to restart a subsystem in place, and neither matters yet.

There is also a subtlety the two-call pattern does not capture: subsystems that allocate **during** their real `init` — the event system creates dynamic arrays, the logger opens a file — put those allocations on the general heap, not in the systems allocator. So the 64 MiB figure describes the state structures, not the engine’s actual footprint.

3 Memory ownership

Game Engine Architecture	GEA §6.2, p.426 Memory Management; GEA §6.2.1.2, p.427 Stack-Based Allocators; GEA §3.3 Data, Code and Memory Layout
Vulkan Specification	VkSpec §11.1 Host Memory; VkSpec §11.2 Device Memory; VkSpec §12.9 Resource Memory Association
Hefest	<code>core/hmemory.c</code> , <code>memory/linear_allocator.c</code> , <code>renderer/vulkan/vulkan_buffer.c</code>

Memory in Hefest is managed at three levels, and each corresponds to a different concept in the sources: a tagged wrapper over the OS allocator, a bump allocator for subsystem state, and explicit device memory on the GPU side.

3.1 Level 1: tagged host allocation

`hallocate(size, tag)` (`core/hmemory.c`) wraps `platform_allocate` and adds a running total per `MEMORY_TAG_*` category — `TEXTURE`, `RENDERER`, `DARRAY`, `GAME`, `SCENE`, and a dozen more. It tracks totals only, not per-allocation metadata, which is why `hfree` requires the caller to pass the size back.

Gregory’s chapter 6.2 argues that engines implement custom allocators for two reasons: speed, and control over fragmentation. Hefest’s wrapper addresses neither directly — it forwards to the platform allocator — but it addresses a third concern Gregory raises separately in [GEA §10.9, In-Game Memory Stats and Leak Detection](#): knowing where the bytes went. The engine can print a tagged breakdown at any time (`get_memory_usage_str`), and the testbed binds it to a debug key. `get_memory_alloc_count` exists specifically so per-frame allocation churn can be watched, which is the diagnostic Gregory recommends for catching accidental allocation inside the game loop.

3.2 Level 2: the linear allocator

`memory/linear_allocator.c` is a bump allocator over a fixed block: `allocate` returns memory + allocated and advances the cursor; `free_all` rewinds it to zero. There is no per-allocation header and no individual free.

This is Gregory’s stack allocator with one feature removed. His [GEA §6.2.1.2, p.427](#) description:

“A pointer to the top of the stack is maintained. All memory addresses below this pointer are considered to be in use, and all addresses above it are considered to be free. [...] Each allocation request simply moves the pointer up by the requested number of bytes.”

The feature Hefest omits is the **marker**. Gregory’s stack allocator hands out an opaque marker representing the current top, and offers a roll-back function that frees everything above a marker in one operation (his Figure 6.1). Hefest has only `free_all`. That is sufficient for the one use it currently has — subsystem state that lives for the whole process — and insufficient for the use Gregory describes next: the **single-frame allocator** [GEA §6.2.1.3, p.442](#), a stack that is cleared at the top of every frame and used for transient per-frame scratch data.

That single-frame allocator is one of the more load-bearing patterns in the book, because it is what lets a game loop do temporary allocation at zero cost. Hefest has the allocator but not the frame discipline; adding it is mostly a matter of adding the marker API and calling `free_all` in `application_run`.

Where it stops. `linear_allocator_allocate` performs no alignment. It returns the next byte, so a subsystem state struct requiring 16-byte alignment gets it only by accident of the preceding sizes. Gregory devotes [GEA §3.3.7](#) to why this is a correctness issue on some architectures and a performance issue on the rest, and Vulkan itself is strict about alignment on the device side ([VkSpec §12.9](#) requires buffer and image memory offsets to be multiples of a driver-reported alignment). The host side of Hefest is currently the loose one. `hmemory.c:71` and `:88` carry matching `// TODO: Memory alignment.` comments — the awareness is there, the implementation is not.

Separately: `linear_allocator_allocate` returns `NULL` on exhaustion and logs an error, but `application_create` does not check the return value before handing the pointer to a subsystem. Exhausting the 64 MiB would produce a null dereference rather than a diagnosable failure.

3.3 Level 3: device memory and the staging pattern

The GPU side is where the specification does the talking. Vulkan does not have “GPU memory” as a single thing; [VkSpec §3.2](#) describes a device as advertising one or more **heaps**, each exposing **memory types** with different properties:

“Device memory is explicitly managed by the application. Each device may advertise one or more heaps, representing different areas of memory. [...] device-local is memory that is physically connected to the device; device-local, host visible is device-local memory that is visible to the host; host-local, host visible is memory that is local to the host and visible to the device and host. On other architectures, there may only be a single heap that can be used for any purpose.” [VkSpec §3.2](#)

That last sentence is the one that matters for this machine. The engine’s most recent commit adapts the Vulkan code to run on non-discrete GPUs, where the integrated-GPU case collapses much of this hierarchy — device-local memory *is* host-visible, and there is no PCIe boundary to cross.

Hefest nonetheless implements the general case, the **staging buffer** pattern (`vulkan_backend.c`, `upload_data_range`):

1. Create a temporary buffer in `HOST_VISIBLE | HOST_COHERENT` memory.
2. `memcpy` the data into it through a mapped pointer.
3. Record and submit a `vkCmdCopyBuffer` from staging into a `DEVICE_LOCAL` buffer.
4. Destroy the staging buffer.

The same four steps, with an added image-layout transition, move texture pixels onto the GPU in `vulkan_renderer_create_texture`. This is the standard shape and it is correct; it is also, on a system where the two memory types coincide, pure overhead that a more adaptive backend would skip. Worth noting as a “structurally right, situationally wasteful” case rather than a defect.

The engine’s device-side allocation strategy is otherwise deliberately blunt: `create_buffers` allocates one vertex buffer and one index buffer of `1 Mi × stride` bytes at start-up and never grows them. Vulkan’s own [VkSpec §11.2.1](#) warns that `maxMemoryAllocationCount` is a real and sometimes small limit, and the accepted answer in the ecosystem is exactly this — allocate a few large blocks and sub-allocate within them. Hefest has the first half (large blocks) without the second (a sub-allocator), so today it can hold exactly one quad’s worth of geometry at offset zero.

4 The frame loop and CPU/GPU pipelining

Game Engine Architecture	GEA §8.2, p.526 The Game Loop; GEA §8.3, p.529 Game Loop Architectural Styles; GEA §8.5, p.534 Measuring and Dealing with Time
Vulkan Specification	VkSpec §3.2 Execution Model; VkSpec §6 Command Buffers; VkSpec §7.3 Fences; VkSpec §7.4 Semaphores; VkSpec §40 Window System Integration
Hefest	<code>core/application.c:150</code> , <code>renderer/renderer_frontend.c:124</code> , <code>renderer/vulkan/vulkan_backend.c:401,521</code>

4.1 Framework, not library

Gregory distinguishes two ways an engine can be structured [GEA §8.3.2, p.530](#). A **library** is called by the application; a **framework** is a partially-constructed application that calls into code the programmer supplies. “The main game loop has been written for us, but it is largely empty. The game programmer can write callback functions in order to ‘fill in’ the missing details.”

Hefest is unambiguously a framework. `entry.h` inside the engine defines `main()`; the game side implements `b8 create_game(game* out_game)` and fills a struct of four function pointers — `initialize`, `update`, `render`, `on_resize`. The testbed never sees the loop. This choice was made early (commit 984b00b, “entry point relocation”) and everything since assumes it.

The loop itself (`application.c:150`) is the shape Gregory describes in [GEA §8.3.1](#), a message pump that services OS events first and then runs one iteration of engine work:

1. `platform_pump_messages()` — drain OS events, which feed the input system, which fires engine events.
2. `clock_update()` — produce `delta_time`.
3. `game->update(delta)` — the testbed moves its camera here.
4. `game->render(delta)` — currently a no-op.
5. `renderer_draw_frame(packet)` — the actual work.
6. `input_update(delta)` — copy current input state to “previous” so `input_was_down()` works next frame.

Gregory’s [GEA §8.5](#) observations about time show up as small concrete decisions: `delta_time` is computed once per iteration and passed down rather than being queried by subsystems, which is what makes it possible to later scale, pause, or fix the timestep from a single place. The frame governor is present but disabled behind a `b8 limit_frames = false` local — the structure for capping the frame rate exists, unengaged.

Note also the ordering comment around `input_update`: it must be the last thing in the iteration, because it is what turns “current” state into “previous” state. That is exactly the kind of ordering constraint that Gregory argues belongs in one visible loop rather than distributed across subsystems.

4.2 Where the GPU changes the picture

Everything above would apply to a software renderer. What Vulkan adds is that step 5 does not actually draw anything — it **records** and **submits** work that the GPU will perform later, possibly while the CPU is already two frames ahead.

[VkSpec §3.2](#) states the model plainly: queue submission commands “should return as soon as the work has been submitted, without waiting for the work to complete”, and “there are no implicit ordering constraints between queue operations on different queues”. Everything that *should* be ordered must be ordered by the application, using two primitives that the specification is careful to distinguish:

Fences “insert a dependency from a queue to the host” [VkSpec §7.3](#). The CPU waits on a fence to learn that GPU work has finished.

Semaphores “insert a dependency between queue operations or between a queue operation and the host” [VkSpec §7.4](#). The GPU waits on a semaphore to learn that other GPU work has finished. The CPU never blocks on a binary semaphore.

Hefest's `begin_frame` / `end_frame` pair implements the standard layered arrangement built from those two:

```
// vulkan_backend.c:401 – begin_frame    (ctx == the file-static vulkan_context)

// Don't run more than max_frames_in_flight ahead of the GPU.
vulkan_fence_wait(&ctx, &ctx.in_flight_fences[ctx.current_frame], UINT64_MAX);

// Returns an index NOW; signals the semaphore when the image is really ready.
vulkan_swapchain_acquire_next_image_index(
    &ctx, &ctx.swapchain, UINT64_MAX,
    ctx.image_available_semaphores[ctx.current_frame], 0, &ctx.image_index);

// This swapchain image may still be in use by an older logical frame.
if (ctx.images_in_flight[ctx.image_index] != 0) {
    vulkan_fence_wait(&ctx, ctx.images_in_flight[ctx.image_index], UINT64_MAX);
}
ctx.images_in_flight[ctx.image_index] = &ctx.in_flight_fences[ctx.current_frame];
vulkan_fence_reset(&ctx, &ctx.in_flight_fences[ctx.current_frame]);

// vulkan_backend.c:521 – end_frame

submit_info.pWaitSemaphores    = &ctx.image_available_semaphores[current_frame];
submit_info.pSignalSemaphores  = &ctx.queue_complete_semaphores[image_index];
VkPipelineStageFlags flags[1] = {VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT};
submit_info.pWaitDstStageMask = flags;

vkQueueSubmit(graphics_queue, 1, &submit_info,
    ctx.in_flight_fences[current_frame].handle);

vulkan_swapchain_present(&ctx, &ctx.swapchain, graphics_queue, present_queue,
    ctx.queue_complete_semaphores[image_index],
    ctx.image_index);
```

Three separate things are being synchronized here, and the code keeps them distinct in a way that is worth spelling out because it is a common source of bugs:

CPU ahead of GPU `in_flight_fences` is sized to `max_frames_in_flight` (2). Waiting on the current frame's fence at the top of `begin_frame` is what stops the CPU from running arbitrarily far ahead and overwriting per-frame resources the GPU is still reading. The fences are created signaled so the first two frames do not deadlock.

Draw after acquire `image_available_semaphores` is also sized to `max_frames_in_flight`. The swapchain image is not ready the moment `vkAcquireNextImageKHR` returns — the call returns an **index** immediately and signals the semaphore when the image is actually available [VkSpec §40.4](#). The submission waits on it at `COLOR_ATTACHMENT_OUTPUT_BIT`, meaning vertex work may start early and only the write to the colour attachment is gated.

Present after draw `queue_complete_semaphores` is sized to `image_count`, which may differ from `max_frames_in_flight`.

That last sizing difference is the non-obvious one, and it is why the code is correct on this machine. The number of swapchain images is chosen by the presentation engine — `min_image_count + 1`, clamped — and has no reason to equal the engine's chosen pipeline depth of 2. If the signal semaphore were indexed by `current_frame` rather than `image_index`, a present operation could end up waiting on a semaphore that a **different** frame's submission will signal. The `images_in_flight` array, which maps swapchain image → the fence of whichever logical frame last used it, closes the remaining hole. Both are noted in the engine's own docs as deliberate, and the most recent commit (“non-discrete GPUs and fixed some underlying issues”) is where this stopped being an assumption and started being handled.

4.3 Resize as a generation counter

Window resize arrives from the OS in the middle of a frame, potentially while the GPU holds references to the swapchain images. Rather than tearing down resources from inside the event handler, `on_resized` increments `framebuffer_size_generation`; the next `begin_frame` notices that it differs from `framebuffer_size_last_generation`, calls `vkDeviceWaitIdle`, recreates the swapchain, and skips the frame.

This is a deferred-mutation pattern that both sources would endorse for different reasons — Gregory because event handlers should not do heavy work mid-frame [GEA §16.8.4](#), the specification because destroying an object still in use by a pending command buffer is undefined behaviour, and the only sanctioned ways to know it is not in use are a fence or a wait-idle [VkSpec §7.8](#).

Where it stops. The loop is single-threaded. Gregory’s [GEA §8.6, p.544](#) — *Multiprocessor Game Loops* — and chapter 4 on parallelism describe the direction every shipping engine goes: a job system, task decomposition, one thread per core. Hefest has a `MEMORY_TAG_JOB` enum value and nothing behind it. Vulkan is designed around this too: [VkSpec §3.6](#) specifies exactly which objects are externally synchronized so that multiple threads can record command buffers into separate pools in parallel, which is the single largest structural win the API offers over its predecessors. Hefest records one primary command buffer per swapchain image on one thread and uses none of it.

`render_packet` currently carries only `delta_time`. The frontend builds a single hardcoded `geometry_render_data` per frame. There is no render queue, no sorting by material or depth, no culling — all of [GEA §11.2.2](#) is unimplemented. The seam for it exists and is marked `// TODO: Refactor packet creation (application.c:186)`.

5 Resources, references and the asset pipeline

Game Engine Architecture	GEA §7.2, p.493 The Resource Manager; GEA §7.2.4, p.509 Resource Lifetime; GEA §11.1.2.6, p.660 Materials; GEA §1.7 Tools and the Asset Pipeline
Vulkan Specification	VkSpec §12.4 Images; VkSpec §12.5 Image Layouts; VkSpec §14 Samplers; VkSpec §17 Descriptor Sets
Hefest	<code>systems/texture_system.c</code> , <code>systems/material_system.c</code> (uncommitted), <code>resources/resource_types.h</code> , <code>assets/materials/*.hmt</code>

5.1 Reference counting by name

Gregory frames the resource manager’s central job as lifetime management [GEA §7.2.4, p.509](#): different assets have different lifetimes — global, per-level, shorter than a level, streamed — and something has to decide when bytes can be reclaimed. His worked example (p. 510) is a table of reference counts incremented as level X is traversed and decremented when it is left, with the unload triggered when a count reaches zero.

`texture_system.c` is a direct implementation of that idea at the smallest useful scale:

```
texture* texture_system_acquire(const char* name, b8 auto_release); // :111
void texture_system_release(const char* name); // :168
```

Internally, a hashtable maps `name` → `texture_reference { reference_count, handle, auto_release }`, and a flat array of texture structs holds the actual records. Acquiring a name that is not yet loaded finds a free slot, loads the PNG through `stb_image`, calls `renderer_create_texture`, and returns the pointer. Releasing decrements; on reaching zero **and** `auto_release` being set, the GPU resources are destroyed and the slot invalidated.

The `auto_release` flag is doing exactly the work Gregory’s lifetime taxonomy implies. A texture acquired with `auto_release = false` behaves as a **global asset** — it stays resident even at zero references. One acquired with `auto_release = true` behaves as a level asset. It is a one-bit version of a policy the book describes at more length, and it is the right one bit.

The material system, currently uncommitted, repeats the pattern one level up: same `material_reference` struct, same hashtable-plus-array, same `INVALID_ID` sentinel filling, same default-object fallback. Two systems with identical shape is where a generic resource manager usually gets factored out, and the code already anticipates this — `material_system.c` opens with a `// TODO: temp: resource_system` block around its direct filesystem include.

5.2 Generations as a staleness signal

Both texture and material carry a generation field alongside `id`. It is incremented whenever the underlying data is reloaded, and set to `INVALID_ID` for a resource that has no valid data yet (including, deliberately, the default texture).

This is the **handle-with-generation** idea Gregory discusses in the context of object references [GEA §16.5](#): a raw pointer cannot tell you that the thing it points at has been replaced, but a `{index, generation}` pair can. In the renderer it lets the Vulkan backend detect that a descriptor set is bound to a stale texture and needs rewriting, rather than rewriting descriptors unconditionally every frame. `vulkan_material_shader.c` compares the texture’s current generation against the one cached in `descriptor_states` and issues a `vkUpdateDescriptorSets` only on mismatch.

That is a small optimization today with one object. It is the correct architecture for many, and it is a good example of a place where the code is already shaped for a scale it has not reached.

5.3 The asset pipeline, in miniature

Gregory’s [GEA §1.7](#) describes the asset conditioning pipeline: DCC tools produce authoring formats, an offline step converts them into engine-native formats, and the runtime loads only the latter. Hefest has a two-stage version of this, split across build-time and run-time:

	Authoring form	Runtime form
Shaders	GLSL under assets/shaders/	SPIR-V, compiled by glslc in post-build.sh
Textures	PNG under assets/textures/	decoded by stb_image, uploaded to a VkImage
Materials	.hmt text file	material_config → material

The shader half is the one that matches the book’s model most closely, and it is Vulkan that forces it: [VkSpec §9.2](#) accepts only SPIR-V, a binary intermediate representation, never GLSL source. The offline compilation step is not an optimization the engine chose, it is a requirement the API imposes — one of the clearer cases where the specification’s contract produces the architecture the book recommends on general principles.

The material half is new and is the engine’s first **data-driven** asset in Gregory’s sense [GEA §15.3](#): a definition that lives in a text file rather than in code.

```
#material file
```

```
version=0.1
name=test_material
diffuse_color=1.0 1.0 1.0 1.0
diffuse_map_name=cobblestone1
```

`load_configuration_file(material_system.c:293)` parses this line by line into a `material_config`. The `version` key is present and currently unused, which is the right thing to write down early — Gregory’s [GEA §7.1.3](#) discussion of asset formats is largely about versioning and the pain of not having it.

Note what the format expresses: a material names its texture rather than containing it. That indirection is what makes the reference-counting layer meaningful — two materials naming `cobblestone1` share one GPU image.

5.4 How this reaches the GPU

The resource concepts above stop at the frontend boundary; from there the specification takes over. A `material` becomes, in Vulkan terms, entries in a descriptor set: [VkSpec §17](#) defines a descriptor set as “an opaque object containing storage for a set of descriptors, where the types and number of descriptors is defined by a descriptor set layout”.

`vulkan_material_shader.c` declares two layouts:

Set 0 — global one `UNIFORM_BUFFER` binding holding projection and view matrices, visible to the vertex stage. One set per swapchain image, written once per frame.

Set 1 — per-object one `UNIFORM_BUFFER` (diffuse colour) plus one `COMBINED_IMAGE_SAMPLER` (the diffuse texture), visible to the fragment stage. One set per object per swapchain image.

This split is not arbitrary — it follows an explicit recommendation in the specification [VkSpec §17.2](#):

“Place the least frequently changing descriptor sets near the start of the pipeline layout, and place the descriptor sets representing the most frequently changing resources near the end. When pipelines are switched, only the descriptor set bindings that have been invalidated will need to be updated...”

Sorting shader inputs by update frequency is a well-known engine-side practice that Gregory describes in [GEA §11.2.3](#) in API-neutral terms; here the API states it as guidance and the engine’s set numbering follows it exactly.

The model matrix — the most frequently changing datum of all, one per draw call — does not go through a descriptor set at all. It is a **push constant** (`vulkan_material_shader.c:280`), which [VkSpec §17.10](#) describes as “a high speed path to modify constant data in pipelines that is expected to outperform memory-backed resource updates”. A small bank of values written directly into the command buffer, no buffer and no descriptor involved. The engine moved to this deliberately in commit `c859db3`, replacing a per-object uniform write.

The result is a three-tier frequency hierarchy that reads cleanly in the code:

Frequency	Mechanism	Contents
Per frame	Descriptor set 0	projection, view
Per object	Descriptor set 1	diffuse colour, diffuse texture + sampler
Per draw	Push constants	model matrix

Where it stops. The material system is uncommitted and not yet wired into `application_create` — the renderer frontend still holds a raw `texture*` for its test quad and swaps it on a debug key. The path `material_system_acquire → load_material → texture_system_acquire` exists in the file but is not on the frame path yet.

There is no resource system above textures and materials, so both reach for `filesystem_open` directly, and paths like `"assets/materials/%s.%s"` are formatted inline with a `//` `TODO: Should be able to be located anywhere`. Gregory’s [GEA §7.1.1](#) argues for exactly the missing piece: a virtual filesystem with logical path roots, so that asset location is a policy rather than a string literal.

Nothing streams. All of [GEA §7.2.5](#) — asynchronous loading, priority queues, loading a level while the previous one plays — is out of scope for a single-threaded engine, and depends on the job system that does not exist yet.

6 Abstraction seams

Game Engine Architecture	GEA §1.6.5, p.43 Platform Independence Layer; GEA §1.6.6, p.44 Core Systems; GEA §1.6.10 The Rendering Engine
Vulkan Specification	VkSpec §3.3 Object Model; VkSpec §40.1 WSI Platform; VkSpec §40.2 WSI Surface
Hefest	platform/platform.h, renderer/renderer_backend.c:5, renderer/renderer_types.inl, renderer/vulkan/vulkan_platform.h

Hefest has three deliberate seams, at three different levels, and they are worth comparing because they are built out of the same C construct for different reasons.

6.1 The platform layer

platform/platform.h declares an OS-agnostic interface — window creation, message pumping, allocation, console output, absolute time, sleep — implemented twice, in platform_win32.c and platform_linux.c, each wrapped in `#if HPLATFORM_*` so both can be handed to the compiler unconditionally.

This is Gregory’s GEA §1.6.5 platform independence layer, and his stated rationale applies directly:

“...some application programming interfaces, like those provided by the operating system, or even some functions in older ‘standard’ libraries like the C standard library, differ significantly from platform to platform; wrapping these functions provides the rest of your engine with a consistent API across all of your targeted platforms.”

Selection is at compile time, not runtime — the seam costs nothing at execution. The Linux implementation (XCB plus X11 plus xcbcommon) arrived late, in 7d2d12b, which is a decent stress test of the abstraction: adding a second platform after the first is where a leaky one shows.

6.2 The renderer frontend/backend split

renderer_backend(renderer_types.inl) is a struct of function pointers: initialize, shutdown, resized, begin_frame, update_global_state, end_frame, update_object, create_texture, destroy_texture. renderer_backend_create(renderer_backend.c:5) fills it based on a renderer_backend_type enum, of which only VULKAN is implemented.

Notice what is **not** in the table. There is no create_pipeline, no begin_render_pass, no bind_descriptor_set. The interface is deliberately coarse-grained: it speaks in terms of frames and objects, not in terms of GPU primitives. That is the correct altitude for this seam, and it is why the frontend can hold projection and view matrices and a texture* without knowing what a VkDescriptorSet is.

The one leak the code admits to is renderer_set_view(mat4), exported as HAPI and marked `// HACK` in the header, so the testbed can push its camera matrix straight into the renderer. The principled version routes it through render_packet instead, and the header says so.

A second, more structural constraint: the function pointers take no self argument. The Vulkan backend keeps its state in a file-static vulkan_context, so the table cannot describe two backend instances. Consistent with the module-static pattern used everywhere else in the engine, and the same trade.

6.3 Vulkan’s own seam

The third seam is not Hefest’s; it is the specification’s, and Hefest sits inside it.

VkSpec §3.3 describes an object model in which every Vulkan object is an opaque handle created from and owned by a parent — instance, then physical device, then logical device, then everything else. VkSpec §40.1 extends this to windowing with a striking constraint: “The Vulkan API does not define any type of platform object.” Window systems are reached only through extensions, each with its own vkCreate*SurfaceKHR entry point, and the specification names the preprocessor guard as the intended mechanism.

Hefest mirrors this exactly, in renderer/vulkan/vulkan_platform.h — a small per-OS shim providing platform_create_vulkan_surface and platform_get_required_extension_names. That file exists be-

cause the specification’s structure demands it, and it sits alongside the engine’s own platform layer doing the same job for the same reason one level down. Two platform abstractions, one imposed by the OS and one imposed by the graphics API, and they do not merge.

6.4 Instrumentation as an engine concern

One seam that Gregory treats as first-class and Hefest implements without naming as such: [GEA §1.6.6](#) lists assertions and logging among the core systems every engine needs, and [GEA §10.1, p.589](#) — *Logging and Tracing* — argues for verbosity levels compiled out of production builds.

`core/logger.h` provides `HFATAL` through `HTRACE` with the lower levels `#defined` away in release builds, and `core/asserts.h` provides `HASSERT` variants. Vulkan’s parallel facility is validation layers, and the engine gates them the same way: `VK_LAYER_KHRONOS_validation` and the `VK_EXT_debug_utils` messenger are enabled only under `_DEBUG` (`vulkan_backend.c`), so release builds carry zero validation cost. [VkSpec §59](#) describes this as the intended usage — validation is opt-in precisely so that shipping applications pay nothing.

Both mechanisms follow the same rule: diagnostics are structural in development builds and absent in shipping ones. That is a decision that has to be made early in an engine’s life, because retrofitting it means touching every call site, and Hefest made it in its second commit.

7 What the sources point at next

The engine is roughly through Part II of *Game Engine Architecture* — chapters 5 through 9, the low-level systems — with chapter 11’s rendering foundations partially in place and everything after it untouched. Read as a map of unexplored territory rather than a to-do list, the sources agree on a rough order.

Area	What the sources describe	Where
Render queue	<code>render_packet</code> carrying a list of mesh-material pairs, sorted to minimise state changes, rather than one hardcoded object. Gregory calls these render packets and treats the sort as the rendering engine’s central runtime job.	GEA §11.2.2
Geometry management	A sub-allocator inside the existing large vertex/index buffers, so that arbitrary meshes can be uploaded rather than one quad at offset zero.	VkSpec §11.2.1 , GEA §6.2.1
Frame allocator	The marker/roll-back API the linear allocator is missing, plus a <code>free_all</code> at the top of the loop — Gregory’s single-frame allocator, which makes transient per-frame allocation free.	GEA §6.2.1.3 , p.442
Alignment	Aligned allocation in <code>hallocate</code> and in the linear allocator; the two matching <code>// TODOs</code> in <code>hmemory.c</code> .	GEA §3.3.7
Resource system	The layer that <code>texture_system</code> and <code>material_system</code> are each partially reimplementing; a virtual filesystem with logical path roots and a generic loader registry.	GEA §7.1 , GEA §7.2
Job system	The largest structural change available. Vulkan is designed for parallel command-buffer recording and specifies exactly which objects are externally synchronized to enable it; the engine currently records on one thread.	GEA §4 , GEA §8.6 , VkSpec §3.6

7.1 A closing observation

The most interesting pattern in this comparison is how often the two sources converge from opposite directions.

Gregory recommends explicit subsystem start-up because C++’s implicit ordering is unpredictable. Vulkan requires explicit sizing because the driver refuses to allocate for you. Neither is talking about the other, and Hefest’s `*_initialize(&size, state)` handshake is what falls out when you take both seriously at once.

Gregory recommends grouping shader inputs by update frequency for performance reasons that predate Vulkan by a decade. The specification recommends the same thing in [VkSpec §17.2](#) for reasons rooted in how descriptor set bindings are invalidated on pipeline switch. The engine’s set 0 / set 1 / push constant hierarchy satisfies both.

Gregory recommends a platform independence layer because operating systems differ. Vulkan mandates one for window system integration because the API deliberately refuses to know what a window is. Hefest ends up with two of them, nested.

This convergence is the useful thing to take from reading the two documents together. A game engine’s structure is not mostly a matter of taste; a surprising amount of it is forced, either by the hardware, by the API contract, or by the ordering constraints of the problem itself. Where Hefest looks like the book, it is usually not because the book was copied — it is because the constraint was the same.